ITA0609-MACHINE LEARNING

CO1-AT2-CONCEPT MAPPING QUESTIONS

NAME: P. VENKATA PRASANTH

Reg: no: 192412491

1. Design a machine learning system for health analysis to predict diseases such as heart disease or diabetes using patient data including age, blood pressure, glucose level, and lifestyle factors. Explain the choice of suitable algorithms and justify their selection. Describe the steps involved in data preprocessing, feature selection, and model training to ensure accurate and reliable predictions

Sol. # ==========

HEALTH DISEASE PREDICTION SYSTEM

No Dataset Upload Required

==========

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ==========
# STEP 1: CREATE SAMPLE HEALTH DATASET
# ==========

np.random.seed(42)
```

```
n = 1000

age = np.random.randint(20, 80, n)
blood_pressure = np.random.randint(80, 180, n)
glucose = np.random.randint(70, 250, n)
bmi = np.random.uniform(18, 40, n)
smoking = np.random.randint(0, 2, n)
exercise = np.random.randint(0, 2, n)

# Disease Rule (for generating labels)
disease = (
    (age > 50) &
    (blood_pressure > 140) &
    (glucose > 140)
).astype(int)

df = pd.DataFrame({
    "Age": age,
    "BloodPressure": blood_pressure,
    "Glucose": glucose,
    "BMI": bmi,
    "Smoking": smoking,
    "Exercise": exercise,
    "Disease": disease
})

print("First 5 Records")
```

```
print(df.head())

# ========================================
# STEP 2: FEATURES & TARGET
# ========================================

X = df.drop("Disease", axis=1)
y = df["Disease"]

# ========================================
# STEP 3: FEATURE SCALING
# ========================================

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ========================================
# STEP 4: TRAIN TEST SPLIT
# ========================================

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)

# ========================================
```

```
# STEP 5: TRAIN RANDOM FOREST MODEL
# ========================================

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# ========================================
# STEP 6: PREDICTION
# ========================================

y_pred = model.predict(X_test)

# ========================================
# STEP 7: EVALUATION
# ========================================

accuracy = accuracy_score(y_test, y_pred)

print("\n========================")
print("MODEL PERFORMANCE")
print("========================")

print(f"Accuracy: {accuracy*100:.2f}%")
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# ========================================
# STEP 8: FEATURE IMPORTANCE
# ========================================

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance")
print(importance)

# ========================================
# STEP 9: NEW PATIENT PREDICTION
# ========================================

new_patient = [[
```

```
    60,     # Age
    150,    # Blood Pressure
    180,    # Glucose
    32,     # BMI
    1,      # Smoking
    0       # Exercise
]]

new_patient_scaled = scaler.transform(new_patient)

prediction = model.predict(new_patient_scaled)

print("\n========================")
print("NEW PATIENT RESULT")
print("========================")

if prediction[0] == 1:
    print("Disease Risk Detected")
else:
    print("No Disease Risk")
```

output

```
First 5 Records
   Age  BloodPressure  Glucose        BMI  Smoking  Exercise  Disease
0   58            113      143  37.213104        0         1        0
1   71            171      132  31.998410        1         1        0
2   48            174      225  34.744674        1         0        0
3   34            151      129  21.521576        1         0        0
4   62            118      148  28.154264        0         0        0

========================
MODEL PERFORMANCE
========================
Accuracy: 99.50%

Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      1.00       185
           1       1.00      0.93      0.97        15

    accuracy                           0.99       200
   macro avg       1.00      0.97      0.98       200
weighted avg       1.00      0.99      0.99       200
```

```
Confusion Matrix:
[[185   0]
 [  1  14]]

Feature Importance
        Feature  Importance
1  BloodPressure    0.378381
0           Age    0.308360
2       Glucose    0.250967
3           BMI    0.050571
4       Smoking    0.006273
5      Exercise    0.005449

========================
NEW PATIENT RESULT
========================
Disease Risk Detected
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(
```

2. In the context of the Find-S algorithm, consider a dataset used to determine whether a loan should be approved based on attributes such as Employment Status, Salary Level, Credit Score, and Loan History. Apply the Find-S algorithm to derive the most specific hypothesis consistent with the positive examples.

Sol.

==========

FIND-S ALGORITHM

Loan Approval Dataset

==========

```
# Training Data
data = [
    ["Employed", "High", "Good", "Clean", "Yes"],
    ["Employed", "Medium", "Good", "Clean", "Yes"],
    ["Unemployed", "Low", "Poor", "Bad", "No"],
    ["Employed", "Medium", "Excellent", "Clean", "Yes"]
]

# Initialize hypothesis
hypothesis = None

print("Applying Find-S Algorithm\n")

for example in data:

    # Consider only positive examples
    if example[-1] == "Yes":

        if hypothesis is None:
            hypothesis = example[:-1].copy()

        else:
            for i in range(len(hypothesis)):
                if hypothesis[i] != example[i]:
                    hypothesis[i] = "?"

        print("Current Hypothesis:", hypothesis)
```

```
print("\n================================")
print("FINAL HYPOTHESIS")
print("================================")
print(hypothesis)
```

output

```
Applying Find-S Algorithm

Current Hypothesis: ['Employed', 'High', 'Good', 'Clean']
Current Hypothesis: ['Employed', '?', 'Good', 'Clean']
Current Hypothesis: ['Employed', '?', '?', 'Clean']

================================
FINAL HYPOTHESIS
================================
['Employed', '?', '?', 'Clean']
```